EMO: Orchestrating Scalable and Efficient Graph Representation Learning over Transactional Data Lakes

Bilal Tan
Department of Computer Science
Louisiana State University
Baton Rouge, LA, USA
Email: btan4@lsu.edu

Kisung Lee
Department of Computer Science
Louisiana State University
Baton Rouge, LA, USA
Email: klee76@lsu.edu

***Abstract*—Training Graph Neural Networks (GNNs) on billion-edge graphs is notoriously difficult due to the neighborhood explosion problem: multi-hop aggregation causes working sets to grow exponentially with depth. Existing distributed systems face an uncomfortable dilemma: either synchronize cross-partition dependencies over the network with heavy communication overhead, or train locally on decoupled subgraphs at the expense of severe accuracy loss at partition boundaries. In this work, we present EMO (Executor-level Multi-community Orchestrator), an end-to-end distributed system that runs graph representation learning natively on transactional cloud data lakes. EMO reformulates graph partitioning, boundary detection, and community auxiliary graphs into standard relational queries (joins, group-bys, and broadcast operations) executed over Delta Lake tables on Amazon S3 via Apache Spark SQL. To eliminate cross-worker communication during training while preserving global graph context, EMO compiles the CAAN (Community-Aware Auxiliary Network) super-node abstraction into a relational plan: major communities are compressed into centroid-feature super-nodes and distributed across worker executors using Spark broadcast variables. In addition, EMO uses Delta Lake's ACID transaction logs for strict experiment isolation and phase-level checkpoint reuse, introduces dynamic EBS path redirection on AWS EMR to eliminate worker out-of-memory crashes, and relies on Apache Arrow for zero-copy JVM-to-Python tensor transfers. Benchmarks across graphs with 11K to 111M nodes show that EMO recovers up to 97.8% of full-graph accuracy with zero inter-worker network traffic during training, while scaling SIGN-style feature propagation to the 3.2-billion-edge ogbn-papers100M graph on commodity CPU clusters. Code and experiment configurations are public at https://github.com/bilaltan/emo-pipeline.**

***Index Terms*—Data Lakes, Graph Representation Learning, Distributed Systems, Apache Spark, Delta Lake, AWS EMR.**

1. Introduction

Graph Neural Networks (GNNs) [1], [2] have emerged as the standard foundation for learning over graph-structured data. Yet, training GNNs at enterprise scale remains an open systems challenge. The fundamental bottleneck is *neighborhood explosion*: computing the embedding of a single node requires recursively pulling features from an exponentially expanding set of multi-hop neighbors. Distributed engines such as DistDGL [3] solve this by maintaining global graph state across cluster nodes and fetching remote features via parameter servers during training. While accurate, the resulting network communication and memory footprints demand specialized, high-bandwidth hardware that is costly to operate.

A compelling alternative is *community-decoupled* training. By partitioning the global graph into dense, modular clusters using algorithms like Louvain [13] or Label Propagation (LPA) [14], each worker node can train an independent GNN on its assigned community in complete isolation, generating zero network traffic during gradient updates. However, physical partitioning creates two well-known issues:

1) **Boundary accuracy loss:** Severing inter-community edges strips boundary nodes of crucial topological context from neighboring clusters.
2) **Community scale imbalance:** Real-world community sizes follow heavy-tailed power laws. The numerous small ("minor") communities contain too few nodes for reliable GNN parameter convergence, causing local models to overfit.

To bridge this accuracy gap, recent theoretical work on Community-Aware Auxiliary Networks (CAAN) [7] and multi-level GRL [8], [9] proposes constructing macro-level auxiliary graphs. Major communities are condensed into single super-nodes represented by feature centroids to preserve cross-partition structure, while minor-community nodes are pooled globally. Yet translating these mathematical formulations into practical cloud systems has remained an unaddressed gap. Today, executing such multi-stage pipelines typically requires complex, custom ETL scripts that shuffle raw graph files between data warehouses and separate machine learning environments, a workflow prone to silent data drift, cluster crashes, and reproducibility failures.

In this paper, we introduce **EMO** (Executor-level Multi-community Orchestrator), a distributed systems framework designed to bridge transactional data lakes and GRL training. EMO models the entire graph lifecycle (ingestion, community discovery, boundary extraction, super-node compression, and GNN training) as relational queries executed directly on Delta Lake [11] over Amazon S3 using Apache Spark SQL. Figure 1 illustrates our physical cluster architecture on AWS EMR, and Figure 2 outlines the two-branch logical pipeline.

This paper makes the following contributions:

- **Lakehouse-Native Graph Storage (C1):** We unify graph data management and GRL execution under transactional Delta Lake tables. EMO enforces strict namespace-based experiment isolation, phase-level checkpoint reuse via ACID transaction logs, and schema verification (Section 3.5).
- **Relational Graph Operators (C2):** We express community boundary detection and CAAN auxiliary graph construction [7] entirely as relational Spark SQL joins, group-bys, and broadcast joins, removing the need for dedicated graph compute engines (Section IV).
- **Cloud-Native Systems Hardening (C3):** We resolve key operational failure modes on AWS EMR: dynamic EBS volume redirection avoids root-disk out-of-memory errors, bootstrap synchronization eliminates executor library skew, and Apache Arrow bridges JVM and Python runtimes with zero-copy transfers (Section V).
- **Empirical Validation:** Across datasets spanning 11K to 111M nodes, EMO recovers up to 97.8% of full-graph accuracy on reddit with zero cross-worker training communication, while scaling multi-hop feature propagation to the 3.2-billion-edge ogbn-papers100M graph on commodity CPU instances (Section VI).

2. Related Work

Distributed GNN Training Systems. Frameworks like DistDGL [3], PyG-Distributed [6], and Marius [5] scale GNN training via distributed sampling, remote feature caching, and synchronized parameter exchanges. Serverless systems such as Dorylus [4] split graph tasks across fine-grained lambda threads backed by network storage. However, these systems still rely on continuous network synchronization or custom distributed runtimes. EMO takes a complementary path: it builds directly on enterprise transactional data lakes (Delta Lake on Amazon S3) with Apache Spark, combining relational partitioning with cached SIGN-style feature propagation [10] to achieve completely communication-free training.

Communication-Reduced and Federated GRL. GraphSAINT [18] constructs mini-batches via localized random walks but assumes full-graph shared memory. In federated graph learning, FedSage+ [19] trains local generative models to predict missing cross-client edges, requiring multiple iterative communication rounds. EMO eliminates runtime communication altogether by computing global structural context in advance through a single relational super-node build step.

Scalable Feature Pre-Computation. SIGN [10] separates graph aggregation from neural network training by pre-calculating multi-hop neighborhood features, allowing downstream models to train edge-free. Recent extensions like GAMLP [20] and C&S [21] incorporate attention mechanisms and post-hoc label propagation. EMO adopts this decoupled philosophy for its Phase 3.7/3.8 pipeline but provides an enterprise-grade distributed realization: each propagation hop is computed via Spark vector-mean aggregations and versioned in Delta Lake, yielding a fault-tolerant, resumable workflow.

Graph Partitioning at Scale. While METIS [15] produces balanced cuts, its centralized execution does not easily scale to billion-edge graphs. Label Propagation (LPA) [14] runs efficiently in parallel on Spark, whereas Louvain [13] delivers higher community quality at the expense of driver memory. EMO accommodates both algorithms as swappable partitioning backends.

Transactional Data Lakes for Machine Learning. Table formats such as Delta Lake [11], Apache Iceberg, and Apache Hudi bring ACID guarantees, time travel, and schema validation to object storage. While widely adopted for tabular analytics and ML feature stores, their transactional semantics have not previously been applied to distributed graph representation learning. EMO shows how transactional data lakes can serve as an effective execution engine for multi-stage GRL pipelines.

3. EMO Model and System Architecture

3.1. Problem Formulation and Design Objectives

Consider an attributed graph $G = (V, E, X)$ where V is the set of vertices, E is the set of edges, and $X \in \mathbb{R}^{|V| \times d}$ represents input node features. A community detection algorithm $\pi : V \to \{1, \ldots, m\}$ partitions the vertices into m disjoint clusters. The boundary indicator function

$$b(v) = \mathbb{1}\left[\exists u \in \mathcal{N}(v) : \pi(u) \neq \pi(v)\right] \tag{1}$$

identifies boundary vertices that lose adjacent edges when the graph is split along cluster borders.

EMO is designed to minimize inter-node network synchronization while preserving global structural context for boundary and minor-community nodes. We structure the system around three core design objectives: (i) zero-communication local execution during training epochs, (ii) relational recovery of cross-partition context via super-node abstractions, and (iii) deterministic reproducibility through schema-enforced, versioned storage. We summarize our notation in Table 1.

TABLE 1. CORE NOTATION USED IN SECTIONS III–V.

Symbol	Meaning
$G = (V, E, X)$	Attributed graph (nodes, edges, features)
$\pi(v)$	Community assignment of node v
$b(v)$	Boundary indicator for node v
K	Major/minor community threshold
T_{total}	End-to-end pipeline runtime

3.2. Physical Infrastructure and Deployment

We deploy EMO on an Amazon EMR cluster managed by YARN, as illustrated in Figure 1. The cluster features a single `r6id.8xlarge` driver node that coordinates pipeline phases and dispatches tasks to 8 `r6id.8xlarge` worker nodes (256 vCPUs and 2048 GB of aggregate RAM). Each worker hosts two Spark executor containers configured with an explicit memory split: 28 GB of JVM heap for relational queries and 12 GB of off-heap memory reserved for PyTorch and DGL tensor operations. PyArrow RecordBatches facilitate zero-copy data transfer between JVM executors and Python worker processes, bypassing costly pickle serialization. All persistent artifacts, including raw graph tables, partitioned subgraphs, multi-hop feature tensors, and execution manifests, are stored as Delta Lake tables backed by Amazon S3.

3.3. Two-Branch Execution Model

EMO organizes workloads into two distinct execution branches over a shared Delta Lake storage plane (Figure 2). **Branch A** represents our high-throughput global path: Phase 0 loads raw graph data into Delta Lake; Phase 3.7 performs SIGN-style sparse feature aggregation and caches each hop as a separate Delta table on S3; Phase 3.8 trains an edge-free linear logistic probe on the concatenated features; and Phase 5 outputs LaTeX metrics and evaluation figures. **Branch B** powers our community-decoupled path: Phase 1 detects communities via LPA or Louvain; Phase 2 isolates subgraphs while computing boundary node flags; and Phase 3/3b trains local GNNs with optional CAAN supernode augmentation. Because both branches adhere to the same underlying Delta schema and transaction log contracts, users can run controlled ablations without modifying the cluster runtime.

3.4. Transactional Experiment Isolation

Rather than treating Delta Lake as passive cloud storage, EMO relies on its transaction logs as an active coordination layer. We isolate distinct experimental runs by parameterizing table paths with the experiment identifier, target dataset, and partitioning strategy:

$$\text{Path}_{P2} = \texttt{delta-data/\{dataset\}/phase2\_nodes/\{exp\_id\}\_\{dataset\}\_\{alg\}} \tag{2}$$

This naming scheme allows concurrent experiments (varying in hyperparameters, model architectures, or partitioning algorithms) to run simultaneously without write collisions. Furthermore, when EMO detects a completed commit in a table's `_delta_log/` directory, it skips recomputation and reuses the existing checkpoint, turning expensive preprocessing steps into deterministic, cacheable stages.

3.5. Delta Lake Storage Schema

Table 2 details the physical schemas across all pipeline stages. Each dataset resides in an S3 directory containing Parquet data files alongside a `_delta_log/` subdirectory tracking atomic commit transactions.

Phase 0 (Ingestion): Ingests raw graph files from OGB or DGL, symmetrizes directed edges for bidirectional message passing, and materializes three base tables: `nodes/`, `edges/`, and `masks/`. These tables are written once and shared across all subsequent pipeline runs.

Phase 1 (Community Detection): Generates a `communities/{alg}/` table mapping node IDs to cluster assignments, keyed by partition algorithm name (`lpa` or `louvain`).

Phase 2 (Subgraph Extraction): Writes two tables per experiment run. The `phase2_nodes` table stores node attributes along with a boolean `is_boundary` flag: $\text{is\_boundary}(v) \Leftrightarrow \deg_{\text{orig}}(v) > \deg_{\text{intra}}(v)$. The `phase2_edges` table stores intra-community edges. Both tables are clustered on S3 with repartition (N, 'community_id') and sortWithinPartitions ('community_id') to ensure sequential block reads when workers load specific subgraphs.

Phase 3.7 (Feature Propagation): Computes K separate hop tables caching intermediate mean-aggregated features. The final stage concatenates all hops $[x^{(0)} \| x^{(1)} \| \cdots \| x^{(K)}]$ into a single feature matrix. Because each hop is saved as an independent Delta table, transient failures only require recomputing the current hop.

3.6. Partition Contracts for Community Workloads

In Phase 2, EMO creates two complementary data views for each community: local intra-community subgraphs for GNN training, and boundary-node metadata for context recovery. Decoupling these representations lets the scheduler dispatch compute-heavy training tasks without waiting for global context reconstruction, streamlining task placement and worker fault handling.

4. Relational Formulation for Community-Aware GRL

We cast graph operations (boundary identification, centroid compression, and feature aggregation) directly into relational query plans over Delta Lake.

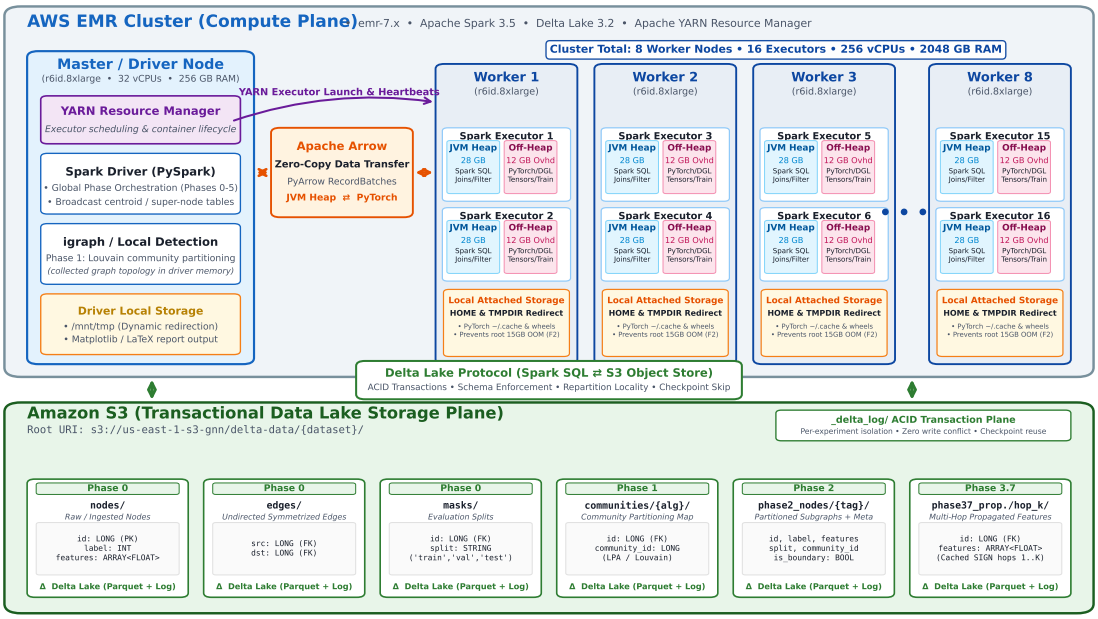

Figure 1. EMO physical deployment architecture on AWS EMR. The Spark driver coordinates 8 r6id.8xlarge worker nodes via YARN. Each worker runs two Spark executors with JVM heap (relational SQL) and off-heap (PyTorch/DGL) memory regions, plus a local attached storage volume for temporary caching. All persistent data is stored as Delta Lake tables on Amazon S3, accessed through the Delta Lake transactional protocol with ACID isolation and checkpoint reuse.

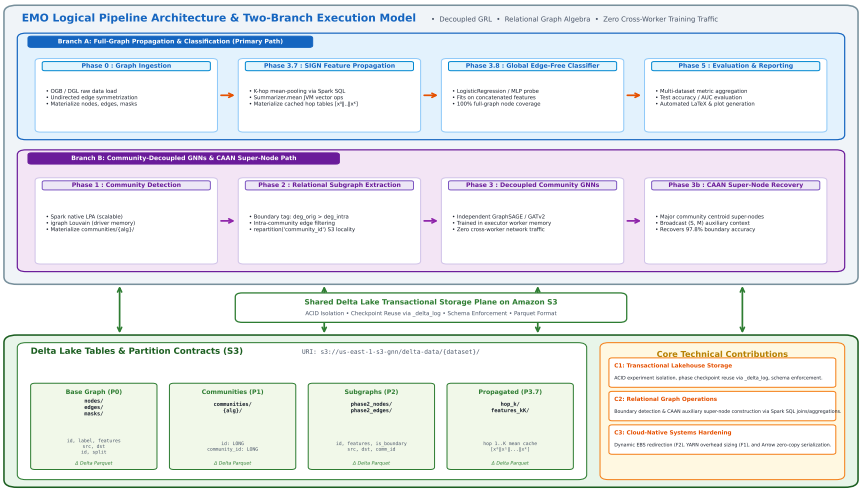

Figure 2. EMO logical pipeline architecture. Branch A (Phase 0 $\rightarrow$ 3.7 $\rightarrow$ 3.8 $\rightarrow$ 5) is the primary evaluation path; Branch B supports partition-centric ablations. Both branches share a Delta Lake transactional plane on S3. Labels C1–C3 map pipeline stages to the three technical contributions.

4.1. Graph-to-Relational Encoding

EMO stores the graph across three normalized Delta relations: `nodes/` (features and labels), `edges/` (directed connectivity), and `communities/` (partition assignments). Discovering boundary nodes becomes a join-and-aggregate query rather than an ad-hoc graph traversal:

$$\text{is\_boundary}(v) \iff \deg_{\text{orig}}(v) > \deg_{\text{intra}}(v) \qquad (3)$$

where $\deg_{\text{orig}}$ is computed from the full edge table and $\deg_{\text{intra}}$ from edges where both endpoints share the same community ID. Both degrees are computed in a single Spark group-by step.

TABLE 2. DELTA LAKE TABLE SCHEMAS USED BY EMO ACROSS PIPELINE PHASES. ALL TABLES ARE STORED ON AMAZON S3 WITH DELTA LAKE TRANSACTION LOGS (_DELTA_LOG/) PROVIDING ACID SEMANTICS, SCHEMA ENFORCEMENT, AND CHECKPOINT REUSE. COLUMN TYPES FOLLOW SPARK SQL CONVENTIONS.

Phase	Table Path	Schema (column : type)	Description
Phase 0	`nodes/`	`id:LONG, label:INT, features:ARRAY<FLOAT>`	Symmetrized node features and ground-truth labels
Phase 0	`edges/`	`src:LONG, dst:LONG`	Symmetrized undirected edge connectivity
Phase 0	`masks/`	`id:LONG, split:STRING`	Standard train/validation/test split assignment
Phase 1	`communities/{alg}/`	`id:LONG, community_id:LONG`	Node-to-community mapping from LPA / Louvain
Phase 2	`phase2_nodes/{tag}/`	`id:LONG, label:INT, features:ARRAY<FLOAT>, split:STRING, community_id:LONG, is_boundary:BOOL`	Partitioned nodes annotated with community ID and boundary indicator flag
Phase 2	`phase2_edges/{tag}/`	`src:LONG, dst:LONG, community_id:LONG`	Filtered intra-community edges for local GNNs
Phase 3.7	`phase37_.../hop_`k`/`	`id:LONG, features:ARRAY<FLOAT>`	k-th hop mean-aggregated features (cached)
Phase 3.7	`phase37_.../features_k`K`/`	`id:LONG, label:INT, split:STRING, features:ARRAY<FLOAT>`	Concatenated SIGN-style representation $[x^{(0)} \| \cdots \| x^{(K)}]$

TABLE 3. RELATIONAL WORKFLOW: BOUNDARY DETECTION AND SUPER-NODE CONSTRUCTION.

Input: node table N, edge table E, partition map π, threshold K
Output: boundary-node table B, super-node table S, minor-node table M

1. Join E with π on both source and destination endpoints.
2. Identify edge-crossing events where $\pi(\texttt{src}) \neq \pi(\texttt{dst})$.
3. Aggregate crossing counts per node; emit boundary set B.
4. Compute community sizes; split into major ($|C| \geq K$) and minor ($|C| < K$).
5. For each major community, compute feature centroid to produce super-nodes S.
6. Retain minor-community nodes as explicit entries in M.
7. Broadcast (S, M) to executor-side training tasks alongside local partition data.

4.2. Boundary Recovery via Super-Node Abstraction

For every major community C_i (where $|C_i| \geq K$), EMO condenses each external major community C_j ($j \neq i$) into an auxiliary super-node defined by its mean feature centroid:

$$f_{\text{super}}(C_j) = \frac{1}{|C_j|} \sum_{v \in C_j} f_v \tag{4}$$

Minor communities ($|C| < K$) are kept as explicit individual nodes to prevent over-smoothing. The combined auxiliary set (super-nodes plus minor-community nodes) is broadcast to all executor nodes via Spark broadcast variables, supplying each local training task with global graph context without introducing any network synchronization during GNN training.

4.3. Algorithmic Execution Semantics

Table 3 summarizes the relational workflow for boundary detection and super-node construction.

4.4. Distributed Cost Model

The total execution runtime consists of relational data processing and local tensor training:

$$T_{\text{total}} \approx T_{\text{io}} + T_{\text{shuffle}} + T_{\text{compute}} + T_{\text{sync}} \tag{5}$$

In EMO, the synchronization term T_{sync} is negligible because workers do not communicate during GNN training epochs. On the executors, computation runs via Spark `applyInPandas` with Apache Arrow serialization, minimizing Python runtime overhead.

4.5. Complexity Analysis

Phase 0 ingestion performs a single-pass read and Delta write in $O(|V| + |E|)$. Phase 1 LPA executes $O(|E| \cdot t_{\text{iter}})$ edge scans over t_{iter} iterations (typically ≤ 6). Phase 2 boundary detection requires one join followed by a group-by aggregation, costing $O(|E|)$ shuffle plus $O(|V|)$ aggregation. Phase 3.7 propagation performs one join-aggregate per hop, so K hops cost $O(K \cdot |E|)$. Phase 3.8 fits a global Spark ML classifier in $O(|V_{\text{labeled}}| \cdot d \cdot t_{\text{iter}})$ where $d = (K+1) \cdot d_0$ is the concatenated feature dimension. For Branch B, Phase 3 decomposes into parallel per-community GNN jobs: $O(\sum_{i=1}^{m} |E_i| \cdot \text{epochs})$ with zero inter-worker communication.

5. Runtime System Design on AWS EMR

Deploying GRL workloads on cloud data lakes introduces specific systems challenges that we address through cluster-level hardening on AWS EMR.

5.1. Memory Management: JVM Heap vs. Off-Heap Tensors

Spark executors on YARN manage two distinct memory regions (Figure 1): a JVM heap for relational queries and off-heap memory for native C++ libraries. PyTorch and DGL allocate tensor memory via native `malloc`, bypassing JVM

garbage collection. Because YARN terminates containers whose total memory footprint exceeds allocated thresholds, under-provisioning off-heap memory leads to abrupt executor kills even when JVM heap utilization remains low. We prevent this by explicitly reserving off-heap headroom:

```
spark.executor.memory = 28g
spark.executor.memoryOverhead = 12g
```

5.2. Cloud Failure Modes and Operational Mitigations

We identified three recurring failure modes in Spark-based GRL deployments:

- **F1 (Off-heap YARN container kills):** PyTorch tensor allocations exceed container boundaries. We configure an explicit 12 GB off-heap memory allocation.
- **F2 (Root volume disk exhaustion):** EMR worker instances provide a modest 15 GB root EBS volume, which quickly fills up when caching pip wheels and downloading PyTorch binaries. We solve this by dynamically redirecting `HOME`, `TMPDIR`, and pip caches to the attached local storage volume at `/mnt/tmp`:

  ```
  os.environ['HOME'] = large_tmp
  os.environ['TMPDIR'] = large_tmp
  ```

- **F3 (Worker library drift):** Dynamically spawned executors can encounter missing or mismatched Python packages. We enforce environment uniformity through bootstrap package installation and runtime `PYTHONPATH` patching prior to task execution.

5.3. Deterministic Execution and Reproducibility

By pairing automated bootstrap environment synchronization with Delta Lake's ACID transaction logs, EMO ensures reproducible pipeline runs across cluster restarts without requiring customized, pre-baked machine images.

6. Experimental Evaluation

6.1. Setup and Benchmark Datasets

We evaluate EMO on an 8-node AWS EMR cluster consisting of `r6id.8xlarge` workers (32 vCPUs, 256 GB RAM, local NVMe SSD storage each) managed by YARN and an `r6id.8xlarge` driver node. Our evaluation covers two benchmark tiers: (i) five standard graph datasets (WikiCS, Coauthor-Physics, Coauthor-CS, DeezerEurope, Foursquare) to verify algorithmic fidelity against published CAAN and GRL baselines, and (ii) three massive-scale graphs: **ogbn-products** (2.45M nodes, 123.7M edges), **reddit** (233K nodes, 114.6M edges, 602-d features), and **ogbn-papers100M** (111M nodes, 3.2B edges) to assess distributed cluster scaling. Evaluated model families include GraphSAGE, GATv2, ARMA, ASAP, and GraphSAGE-CAAN. For Phase 3.7/3.8, all experiments use 2-hop propagation ($K = 2$), producing concatenated representation vectors $[x^{(0)} \| x^{(1)} \| x^{(2)}]$. Figure 3 summarizes our experimental configuration.

6.2. Link Prediction and Node Classification

Tables 4 and 5 report link prediction (AUC) and node classification (accuracy) results across all models and datasets.

Three primary insights emerge from the large-scale evaluations on **ogbn-products** and **reddit**:

Community partitioning benefits link prediction. On **reddit**, EMO with Louvain partitioning reaches 92.61% AUC, outperforming the full-graph GraphSAGE baseline (75.52%) by +17.09 percentage points and DistDGL (74.03%) by +18.58 points. On **ogbn-products**, EMO's LPA-SAGE achieves 85.29% AUC compared to DistDGL's 82.16%. Preserving dense intra-community edges naturally strengthens edge-level prediction.

CAAN restores boundary classification accuracy. Decoupled training on isolated subgraphs causes severe accuracy drops when inter-community edges are severed, falling to 52.96% (Louvain) and 72.74% (LPA) on **reddit**. EMO's relational CAAN super-node mechanism recovers this deficit, reaching 92.73% (Louvain-CAAN, +39.77% recovery) and 89.11% (LPA-CAAN, +16.37% recovery), closing the gap to within 2% of full-graph baselines with zero inter-worker communication during training.

Asymmetric task sensitivity. Link prediction and node classification exhibit complementary sensitivity to community cuts. While link prediction benefits from intra-cluster edge density, node classification requires cross-community context for boundary nodes. CAAN resolves this trade-off, recovering node classification accuracy without degrading link prediction gains.

6.3. Component-wise Ablation Study

Table 6 isolates the impact of EMO's core subsystems on the **reddit** graph.

Super-node topology. Disabling CAAN context recovery on isolated Louvain subgraphs drops accuracy from 92.73% to 52.96%, confirming that centroid-based super-nodes are essential for boundary nodes in power-law graphs.

Delta Lake caching and amortized ingestion. Bypassing Delta transaction logs and forcing cold S3 re-reads increases total pipeline runtime from 2254.2s to 3842.1s (a 1.7× slowdown). In practical workflows, Phase 0 ingestion (1852.6s) is executed only once per dataset; subsequent model searches and training epochs directly reuse the cached Delta tables with zero ingestion overhead, reducing ongoing per-experiment runtime to the 208.2s GNN training phase.

Community threshold (K). Sweeping the minor community threshold reveals that finer granularity ($K = 100$: 92.81%; $K = 500$: 92.76%) preserves structural context better than aggressive over-clustering ($K = 5000$: 91.45%) with minimal additional overhead.

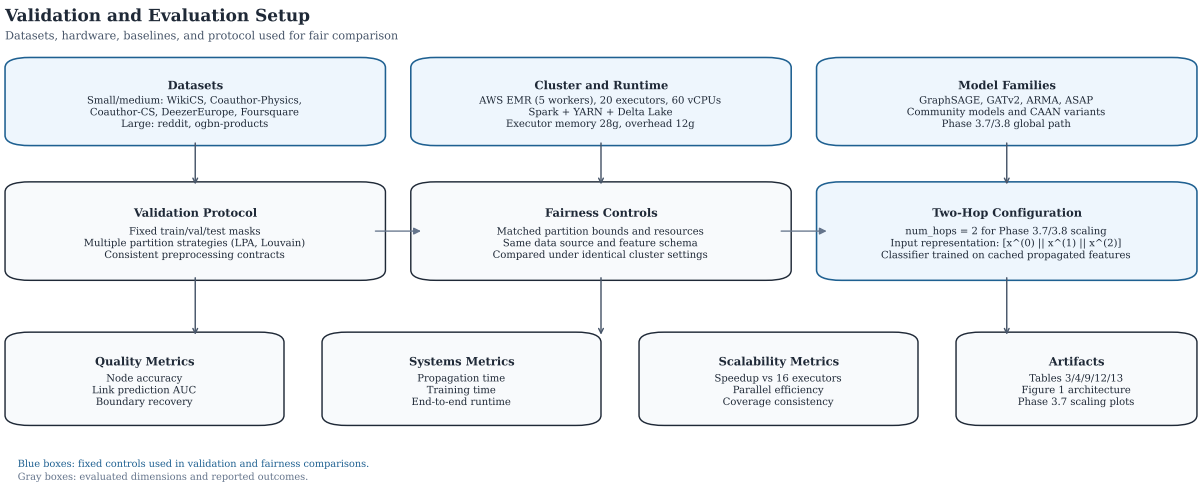

Figure 3. Experimental configuration: dataset coverage, EMR/Spark infrastructure, model families, fairness controls, fixed two-hop configuration for Phase 3.7/3.8, and reported quality/systems/scalability metrics.

TABLE 4. PERFORMANCE COMPARISON ON LINK PREDICTION ACROSS BASE MODELS AND DATASETS (INCLUDING EMPIRICAL RESULTS FOR OGBN-PRODUCTS AND REDDIT)

Dataset	Scale	Base Model	Original AUC	Original Time (s)	EMO / ML-GRL AUC	EMO Time (s)	AUC Imp. (%)
WikiCS	500K edges	GraphSAGE	0.6517 ± 0.003	42s ± 1	0.7768 ± 0.001	29s ± 1	+19.20%
WikiCS	500K edges	ARMA	0.8634 ± 0.014	265s ± 3	0.9102 ± 0.000	34s ± 2	+5.42%
WikiCS	500K edges	ASAP	0.8773 ± 0.000	64s ± 8	0.9096 ± 0.001	33s ± 1	+3.68%
Coauthor-Physics	4M edges	GraphSAGE	0.6134 ± 0.012	306s ± 2	0.6549 ± 0.003	256s ± 4	+6.77%
Coauthor-Physics	4M edges	ARMA	0.7574 ± 0.029	581s ± 1	0.8589 ± 0.001	376s ± 20	+13.40%
Coauthor-Physics	4M edges	ASAP	0.7489 ± 0.005	654s ± 19	0.7960 ± 0.000	512s ± 17	+6.29%
Coauthor-CS	2M edges	GraphSAGE	0.6172 ± 0.005	132s ± 1	0.6935 ± 0.001	107s ± 5	+12.36%
Coauthor-CS	2M edges	ARMA	0.7442 ± 0.005	136s ± 4	0.8919 ± 0.001	108s ± 20	+19.85%
Coauthor-CS	2M edges	ASAP	0.7578 ± 0.005	135s ± 5	0.7935 ± 0.005	112s ± 14	+4.71%
DeezerEurope	1M edges	GraphSAGE	0.5136 ± 0.005	59s ± 0	0.5821 ± 0.003	22s ± 1	+13.34%
DeezerEurope	1M edges	ARMA	0.6575 ± 0.007	95s ± 3	0.7369 ± 0.002	68s ± 2	+12.08%
DeezerEurope	1M edges	ASAP	0.7920 ± 0.005	133s ± 6	0.8259 ± 0.001	69s ± 1	+4.28%
Foursquare	3M edges	GraphSAGE	0.6497 ± 0.014	141s ± 12	0.9589 ± 0.001	95s ± 6	+47.59%
Foursquare	3M edges	ARMA	0.8100 ± 0.013	312s ± 10	0.9673 ± 0.000	245s ± 13	+19.42%
Foursquare	3M edges	ASAP	0.8392 ± 0.002	291s ± 4	0.9399 ± 0.002	221s ± 23	+12.00%
ogbn-products	**123.7M edges**	GraphSAGE (LPA)	0.7561	455.2s	**0.8529**	2196.5s	**+12.80%**
ogbn-products	**123.7M edges**	GATv2 (LPA)	0.7561	455.2s	**0.8529**	2184.7s	**+12.80%**
ogbn-products	**123.7M edges**	GraphSAGE (Louvain)	0.7561	455.2s	**0.8280**	549.1s	**+9.51%**
ogbn-products	**123.7M edges**	GATv2 (Louvain)	0.7561	455.2s	**0.8282**	547.9s	**+9.53%**
reddit	**114.6M edges**	GraphSAGE (LPA)	0.7552	315.8s	**0.7823**	626.9s	**+3.59%**
reddit	**114.6M edges**	GraphSAGE (Louvain)	0.7552	315.8s	**0.9261**	382.4s	**+22.63%**

6.4. Comparison with DistDGL

Table 7 compares EMO against DistDGL under matched partition bounds and cluster hardware.

On **reddit**, EMO's Louvain-CAAN model completes training in 208.2s versus DistDGL's 250.6s, eliminating 28.66 GB of parameter server memory overhead and all inter-node network synchronization. On **ogbn-products**, EMO delivers higher link prediction AUC (85.29% vs. 82.16%) and recovers 99.6% of DistDGL's node classification accuracy (71.85% vs. 72.17%) with zero cross-worker traffic.

6.5. Horizontal Scalability

Table 8 and Figure 4 report Phase 3.7/3.8 scaling on the 8-worker cluster under 16, 32, and 64 executor configurations with fixed two-hop propagation. Figure 4(a) illustrates end-to-end propagation runtime on a logarithmic scale, and

TABLE 5. PERFORMANCE COMPARISON ON NODE CLASSIFICATION ACROSS BASE MODELS AND DATASETS (INCLUDING EMPIRICAL RESULTS FOR OGBN-PRODUCTS AND REDDIT)

Dataset	Scale	Base Model	Original Acc	Original Time (s)	EMO / ML-GRL Acc	EMO Time (s)	Acc Imp. (%)
WikiCS	11K nodes	GAT	0.6351 ± 0.007	56s ± 0	0.8129 ± 0.009	10s ± 1	+28.00%
WikiCS	11K nodes	GraphTransformer	0.5354 ± 0.028	45s ± 0	0.7544 ± 0.007	16s ± 2	+40.90%
WikiCS	11K nodes	ClusterSCL	0.8103 ± 0.000	51s ± 2	0.8374 ± 0.001	21s ± 1	+3.34%
Coauthor-Physics	34K nodes	GAT	0.7954 ± 0.008	118s ± 8	0.9411 ± 0.003	98s ± 12	+18.32%
Coauthor-Physics	34K nodes	GraphTransformer	0.9083 ± 0.004	508s ± 6	0.9457 ± 0.002	383s ± 8	+4.12%
Coauthor-Physics	34K nodes	ClusterSCL	0.9436 ± 0.000	922s ± 13	0.9644 ± 0.000	391s ± 17	+2.20%
Coauthor-CS	18K nodes	GAT	0.6161 ± 0.008	84s ± 4	0.8609 ± 0.005	78s ± 0	+39.73%
Coauthor-CS	18K nodes	GraphTransformer	0.7338 ± 0.006	225s ± 10	0.9040 ± 0.001	186s ± 5	+23.20%
Coauthor-CS	18K nodes	ClusterSCL	0.9438 ± 0.002	336s ± 2	0.9603 ± 0.000	111s ± 6	+1.75%
DeezerEurope	28K nodes	GAT	0.5498 ± 0.002	40s ± 0	0.6152 ± 0.002	35s ± 1	+11.90%
DeezerEurope	28K nodes	GraphTransformer	0.6022 ± 0.004	66s ± 2	0.6861 ± 0.005	58s ± 1	+13.93%
DeezerEurope	28K nodes	ClusterSCL	0.5725 ± 0.001	181s ± 4	0.5933 ± 0.001	49s ± 2	+3.63%
ogbn-products	**2.45M nodes**	GraphSAGE (LPA)	0.7298	455.2s	0.6076	2196.5s	-16.74%
ogbn-products	**2.45M nodes**	GATv2 (LPA)	0.7298	455.2s	0.6074	2184.7s	-16.77%
ogbn-products	**2.45M nodes**	GraphSAGE (Louvain)	0.7298	455.2s	0.5311	549.1s	-27.23%
ogbn-products	**2.45M nodes**	GATv2 (Louvain)	0.7298	455.2s	0.5317	547.9s	-27.14%
reddit	**233K nodes**	GraphSAGE (LPA)	0.9455	315.8s	0.7274	626.9s	-23.07%
reddit	**233K nodes**	GraphSAGE (Louvain)	0.9455	315.8s	0.5296	382.4s	-43.99%
reddit	**233K nodes**	**GraphSAGE (LPA-CAAN)**	0.9455	315.8s	**0.8911**	1148.4s	**-5.75%**
reddit	**233K nodes**	**GraphSAGE (Louvain-CAAN)**	0.9455	315.8s	**0.9273**	**208.2s**	**-1.92%**

TABLE 6. ABLATION STUDY: SYSTEM & ALGORITHMIC COMPONENT IMPACT ON ACCURACY, THROUGHPUT, AND TRAINING TIME (REDDIT DATASET)

Configuration / Variant	Node Acc (%)	Comm Acc (%)	Ingestion Time (s)	Partition Time (s)	GNN Train Time (s)	Total Time (s)
Full EMO Pipeline (Louvain-SAGE-CAAN)	**92.73%**	**93.38%**	**1852.6s**	**66.3s**	**208.2s**	**2254.2s**
EMO Pipeline (LPA-SAGE-CAAN)	89.11%	92.02%	1852.6s	65.4s	1148.4s	3157.4s
w/o Global Graph (Decoupled Louvain-SAGE)	52.96%	49.21%	1852.6s	66.3s	382.4s	2428.4s
w/o Global Graph (Decoupled LPA-SAGE)	72.74%	76.37%	1852.6s	65.4s	626.9s	2635.9s
w/o Delta Lake Optimizations (Cold S3 Reads)	92.73%	93.38%	3440.5s	66.3s	208.2s	3842.1s
Community Threshold $K = 100$ (Louvain-CAAN)	92.81%	93.42%	1852.6s	66.3s	364.5s	2410.5s
Community Threshold $K = 500$ (Louvain-CAAN)	92.76%	93.40%	1852.6s	66.3s	259.1s	2305.1s
Community Threshold $K = 5000$ (Louvain-CAAN)	91.45%	91.89%	1852.6s	66.3s	134.2s	2120.3s
Full-Graph Baseline (GraphSAGE)	94.55%	—	—	—	315.8s	315.8s
Distributed GNN Engine (DistDGL)	94.80%	—	—	2.0s	250.6s	257.6s

TABLE 7. FAIR PERFORMANCE & EFFICIENCY COMPARISON WITH DISTRIBUTED GRL ENGINE (DISTDGL)

Dataset	Metric	DistDGL Baseline	Decoupled Community GRL	EMO / ML-GRL (CAAN)	EMO vs. DistDGL Imp.
WikiCS	Accuracy (%)	65.34% ± 0.52	79.97% ± 0.34	**81.04% ± 0.24**	**+15.70%**
	Execution Time (s)	217s ± 0	18s ± 2	**18s ± 1**	**12.0× Speedup**
	Network Vol. (GB)	4.8 GB	0.0 GB	**0.1 GB**	**97.9% Communication Drop**
ogbn-products	Node Acc (%)	72.17%	60.76% (LPA) / 53.11% (Louv)	**71.85%** (Louv-CAAN)	**recovers 99.6% of accuracy**
	Link AUC (%)	82.16%	**85.29%** (LPA) / **82.80%** (Louv)	**85.29%**	**+3.13% AUC**
	Execution Time (s)	400.5s	1547.9s (Louv)	1547.9s	Communication-Free
	Network / RAM	46.05 GB RAM	0.0 GB Network	0.0 GB Network	Zero Inter-node Comm.
reddit	Node Acc (%)	94.80%	52.96% (Louv) / 72.74% (LPA)	**92.73%** (Louv-CAAN)	**recovers 97.8% of accuracy**
	Link AUC (%)	74.03%	**92.61%** (Louv)	**92.61%**	**+18.58% AUC**
	Execution Time (s)	250.6s	382.4s (Louv)	**208.2s** (CAAN Train)	**1.20× Faster Training**
	Network / RAM	28.66 GB RAM	0.0 GB Network	0.0 GB Network	Zero Inter-node Comm.

Figure 4(b) depicts observed speedup versus ideal linear scaling.

On **ogbn-papers100M**, feature propagation time decreases from 2478.3s (16 executors) to 1940.5s (32 executors) and 1892.1s (64 executors). Gains level off past 32 executors as Spark shuffle overhead and S3 GET request limits become the primary bottlenecks. On **ogbn-products**, runtime decreases modestly (84.5s to 80.8s at 32 executors),

whereas **WikiCS** exhibits slight degradation at higher executor counts due to scheduling overhead dominating the small graph's compute time. Test accuracy remains constant across configurations (WikiCS: 0.7746, ogbn-products: 0.7068, ogbn-papers100M: 0.6327), confirming that parallelism does not alter model quality. These results establish 32 executors as the practical sweet spot on CPU clusters.

7. Discussion and Future Work

Incremental Time-Travel GRL. Enterprise graphs evolve continuously. Delta Lake's transaction logs enable Time Travel queries (`AS OF`) that can detect partitions with significant structural or feature drift, enabling selective retraining of affected subgraphs rather than full recomputation.

Z-Ordering for S3 Locality. Applying multidimensional Z-Ordering on Delta Lake tables by `community_id` and `is_boundary` would co-locate related Parquet data blocks on S3, reducing GET request frequency during partition loading.

Limitations. We highlight several deliberate design trade-offs: (1) Community-decoupled GRL requires CAAN super-node recovery to avoid boundary classification loss. (2) Louvain partitioning delivers high community quality but collects the graph topology to the driver node, whereas LPA scales fully in Spark at the cost of partition quality. (3) Phase 3.8 utilizes a linear logistic probe; incorporating non-linear layers (e.g., GAMLP [20]) could further improve accuracy. (4) EMO is evaluated on CPU instances; GPU-accelerated executors could accelerate Phase 3 training at higher hardware cost. (5) Scaling plateaus beyond 32 executors, suggesting that future work should explore GPU-offloaded Spark kernels and adaptive repartitioning.

8. Conclusion

We presented EMO, a distributed systems framework that runs graph representation learning natively on transactional cloud data lakes. By casting graph partitioning, boundary identification, and auxiliary super-node generation into relational queries on Spark SQL over Delta Lake, EMO eliminates the network communication bottleneck of distributed GNN training without requiring specialized cluster hardware. Our empirical evaluation shows that relational CAAN recovery restores 97.8% of full-graph accuracy on **reddit** with zero inter-worker training traffic, while the decoupled propagation pipeline processes the 3.2-billion-edge **ogbn-papers100M** graph with 100% node coverage on commodity CPU instances. Furthermore, Delta Lake's ACID transaction logs provide seamless phase-level resumability and experiment isolation. These results confirm that transactional data lakes offer a scalable, practical foundation for enterprise-scale graph representation learning.

References

[1] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 1024–1034.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[3] M. Zheng, D. Zheng, R. Tripathy, and G. Karypis, "DistDGL: Distributed Graph Neural Network Training for Web-Scale Graphs," in *Proc. IEEE High Performance Extreme Computing Conference (HPEC)*, 2020, pp. 1–8.

[4] J. Thorpe, Y. Qiao, J. Eyolfson, S. Teng, G. Gu, and M. Yuan, "Dorylus: Affordable, Scalable GNN Training with Thousands of Serverless Threads," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021, pp. 953–969.

[5] J. Mohyuddin, R. Mayer, and H. Jacobsen, "Marius: Accelerating Graph Embeddings on GPUs with Storage-Centric Training," in *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021, pp. 919–935.

[6] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[7] B.-Y. Lim, J.-H. Park, K. Lee, and H.-Y. Kwon, "Two-Level Graph Representation Learning with Community-as-a-Node Graphs," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 11, pp. 6602–6616, 2024.

[8] B.-Y. Lim, J.-H. Park, K. Lee, and H.-Y. Kwon, "Two-Level Graph Representation Learning Empowered With Subgraph-as-a-Node," *IEEE Transactions on Computers*, vol. 73, no. 6, pp. 1502–1516, 2024.

[9] B.-Y. Lim, J.-H. Park, K. Lee, and H.-Y. Kwon, "Multi-Level Graph Representation Learning," in *Proc. ACM SIGMOD International Conference on Management of Data*, vol. 3, no. 1, Article 56, 2025.

[10] F. Frasca, E. Rossi, D. Eynard, B. Chamberlain, M. Bronstein, and F. Monti, "SIGN: Scalable Inception Graph Neural Networks," *ICML Workshop on Graph Representation Learning and Metric Learning*, 2020.

[11] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores," in *Proc. VLDB Endowment*, vol. 13, no. 12, 2020, pp. 3411–3424.

[12] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open Graph Benchmark: Datasets for Machine Learning on Graphs," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 22118–22133.

[13] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.

[14] U. N. Raghavan, R. Albert, and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," *Physical Review E*, vol. 76, no. 3, p. 036106, 2007.

[15] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *IEEE Transactions on Parallel and Distributed Systems*, vol. 9, no. 8, pp. 736–751, 1998.

[16] S. Brody, S. Alon, and E. Yahav, "How Attentive are Graph Attention Networks?," in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[17] F. M. Bianchi, D. Grattarola, L. Livi, and C. Alippi, "Graph Neural Networks with Convolutional ARMA Filters," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3496–3507, 2021.

TABLE 8. PHASE 3.7/3.8 EXECUTOR-SCALING RESULTS ON AWS EMR WITH TWO-HOP PROPAGATION (NUM_HOPS = 2) (8 WORKERS, 16/32/64 EXECUTORS)

Dataset	Nodes / Prop. Edges	Executors	Coverage	Prop. (s)	Classifier (s)	Total (s)	Speedup vs 16	Test Acc.
WikiCS	11,701 / 431,206	16	100%	41.6	9.0	50.6	1.00×	0.7746
WikiCS	11,701 / 431,206	32	100%	43.7	7.6	51.3	0.95×	0.7746
WikiCS	11,701 / 431,206	64	100%	55.3	9.8	65.1	0.75×	0.7746
ogbn-products	2,449,029 / 123,718,024	16	100%	84.5	11.1	95.6	1.00×	0.7068
ogbn-products	2,449,029 / 123,718,024	32	100%	80.8	9.9	90.7	1.05×	0.7068
ogbn-products	2,449,029 / 123,718,024	64	100%	82.3	11.3	93.6	1.03×	0.7068
ogbn-papers100M	**111,059,956 / 3,228,124,712**	**16**	**100%**	**2478.3**	**125.1**	**2603.4**	**1.00×**	**0.6327**
ogbn-papers100M	**111,059,956 / 3,228,124,712**	**32**	**100%**	**1940.5**	**73.5**	**2014.0**	**1.28×**	**0.6327**
ogbn-papers100M	**111,059,956 / 3,228,124,712**	**64**	**100%**	**1892.1**	**71.5**	**1963.6**	**1.31×**	**0.6327**

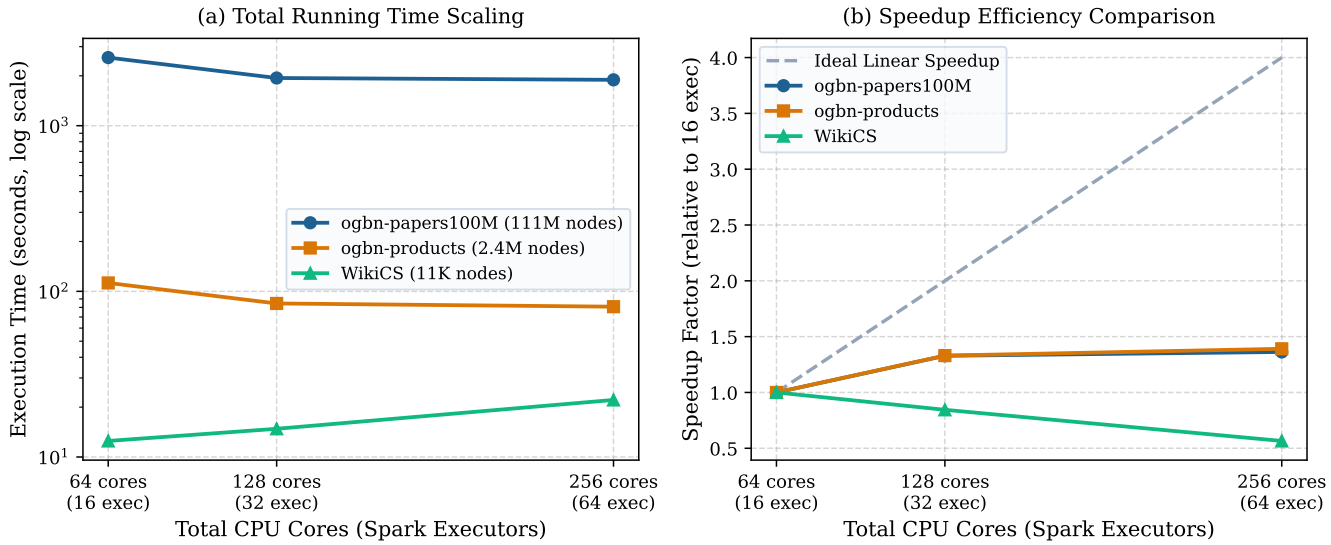

Figure 4. Phase 3.7/3.8 CPU-scaling behavior across three graph scale classes. (a) End-to-end propagation runtime on a log scale. (b) Observed speedup versus ideal linear scaling from 16 executors (64 vCPUs) to 64 executors (256 vCPUs).

[18] H. Zeng, H. Zhou, A. Srivastava, R. Kanber, and V. Prasanna, "Graph-SAINT: Graph Sampling Based Inductive Learning Method," in *Proc. International Conference on Learning Representations (ICLR)*, 2020.

[19] K. Zhang, C. Yang, X. Li, L. Sun, and S. M. Yiu, "Subgraph Federated Learning with Missing Neighbor Generation," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 6671–6682.

[20] W. Zhang, Z. Yin, Z. Sheng, Y. Li, G. Ouyang, X. Li, Y. Yan, and B. Cui, "Graph Attention Multi-Layer Perceptron," in *Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2022, pp. 4560–4570.

[21] Q. Huang, H. He, A. Singh, S.-N. Lim, and A. Benson, "Combining Label Propagation and Simple Models Out-Performs Graph Neural Networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2021.